

Hop6: Small-World Graph Routing for Transformer Inference

Architecture, a Train/Test Mismatch Case Study, and Proposed Extensions

Abhay

September 25, 2026

Abstract

Hop6 is a transformer inference architecture that treats a pretrained model’s layers as nodes in a small-world graph and routes each token along a short path through that graph, rather than through every layer sequentially. Short paths reduce the number of layers executed per token, at the cost of skipping intermediate transformations the model was trained to expect. A lightweight *bridge network* is trained to correct the resulting distortion at each skip (“worm-hole”) edge. This paper documents the architecture, reports a concrete debugging case study in which two root causes of degenerate (gibberish) output were identified and fixed – non-deterministic graph topology between training and inference, and a distribution mismatch between the bridge’s training targets and its deployment-time inputs – and outlines three proposed extensions (confidence-based early halting, information-bottleneck routing, and graph-attached long-term memory), each explicitly situated against existing prior work rather than presented as unprecedented. No extension has been benchmarked at the time of writing; this is a design and diagnosis document, not a results paper.

Contents

1	Introduction	2
2	Architecture	2
2.1	Layer graph and routing	2
2.2	The bridge network	2
2.3	Bridge training, as specified	3
3	Case Study: Diagnosing Degenerate Output	3
3.1	Root cause 1: Non-deterministic graph topology	3
3.2	Root cause 2: Train/test distribution mismatch	3
3.3	Isolation testing	4
3.4	Status	4
4	Proposed Extensions	4
4.1	Confidence-based early halting	4
4.2	Information-bottleneck routing	5
4.3	Graph-attached long-term memory	5
4.4	Combined resource cost, unverified	5
5	Limitations	6

1 Introduction

Standard transformer decoding executes every layer, for every token, sequentially. This is simple and well-optimized, but it treats all tokens as equally difficult and does not exploit any redundancy between a model’s layers. Hop6 instead models the layer stack as a graph: layers are nodes, consecutive layers are connected by “sequential” edges, and a small number of additional long-range “wormhole” edges connect non-adjacent layers. A token’s path through the model is a short path through this graph rather than a walk over every node, so a token may, for instance, jump from layer 3 directly to layer 9 if the graph provides that edge.

The motivation is memory: on hardware where the full model does not fit resident in VRAM, only the layers on a token’s chosen path need to be paged in, so peak memory is bounded by path length rather than model depth.

The cost is quality. A layer expects its input to be the output of the *immediately preceding* layer, produced through the transformations the model was trained with. A wormhole edge violates this: the destination layer receives a hidden state that never passed through the skipped layers. The **bridge network** exists to correct this – a small learned function sitting at each wormhole edge that maps the distorted hidden state back toward what the destination layer expects.

This paper has three parts. Section 2 describes the Hop6 and bridge architecture as implemented. Section 3 is a case study: the system was producing degenerate output, and this section documents the diagnostic process, the two root causes found, and the fixes applied. Section 4 proposes three extensions to the routing mechanism and, for each, distinguishes the part that is prior art from the part that would be specific to Hop6’s graph structure.

2 Architecture

2.1 Layer graph and routing

Let a pretrained transformer have L layers. Hop6 constructs a graph $G = (V, E)$ where $V = \{1, \dots, L\}$ and E consists of:

- **Sequential edges** $(i, i+1)$ for all i , preserving the option of standard layer-by-layer execution;
- **Wormhole edges** (i, j) with $j > i + 1$, added according to a small-world construction (a small number of long-range shortcuts added to an otherwise local/sequential graph, in the spirit of Watts–Strogatz small-world graphs).

For a given input, Hop6 computes a short path from layer 1 to layer L through G (e.g. via shortest-path search over a hop-count or latency cost), and executes only the layers on that path.

2.2 The bridge network

At each wormhole edge (i, j) used, the hidden state h_i produced by layer i is not the input layer j was trained to expect (it was trained to expect h_{j-1} , the output of the immediately preceding layer). The bridge network $B_{i,j}$ is a small MLP applied at that edge, using a gated residual form:

$$\hat{h} = h_i + \sigma(g) \cdot B_{i,j}(h_i) \tag{1}$$

where g is a learned scalar gate and σ is the sigmoid function. The gate is initialized at $g_0 = -6.0$, so $\sigma(g_0) \approx 0.0025$: at initialization the bridge contributes negligible correction, and the architecture degrades gracefully to raw (uncorrected) skip-routing until the bridge has learned something useful.

2.3 Bridge training, as specified

Training proceeds in two phases:

1. **Calibration.** Run the full, unmodified model (all L layers, sequentially) over a text corpus and record every layer-boundary hidden state. This produces the *target* distribution the bridge should map toward.
2. **Paired MSE training.** Run the same inputs through Hop6’s actual skip-path (wormholes active, bridge disabled during data collection) and record the hidden state arriving at each wormhole destination – the bridge’s *input* distribution. Train $B_{i,j}$ to minimize

$$\mathcal{L} = \mathbb{E} \left[\|\hat{h}_j - h_{j-1}^{\text{full}}\|_2^2 \right] \quad (2)$$

where h_{j-1}^{full} is the full-model hidden state at the position layer j expects as input (Section 3.2 discusses why this indexing choice matters).

3 Case Study: Diagnosing Degenerate Output

At an early stage, Hop6 with the bridge enabled produced gibberish output and perplexity evaluated to ∞ on held-out text. Three candidate root causes were considered: (1) an untrained or mis-initialized bridge network, (2) a KV-cache / paging alignment bug, (3) non-deterministic graph topology between training and inference. Root causes (1) partial and (3) full were confirmed; (2) remains open at the time of writing.

3.1 Root cause 1: Non-deterministic graph topology

The graph construction function used an unseeded random number generator to place wormhole edges. Every construction of the inference engine – whether for training data collection or for inference – produced a *different* wormhole topology. Consequently the bridge was trained on corrections for one graph’s edges and deployed against a different graph’s edges at inference time. No amount of additional training data could have fixed this, since the learned correction was, by construction, for the wrong routing problem at inference time.

Fix. The graph constructor now accepts an explicit seed or an exact edge list. The training script serializes the edge list used during calibration into a metadata file, and the inference entry point loads and reconstructs the graph from that exact edge list rather than resampling.

3.2 Root cause 2: Train/test distribution mismatch

Independent of topology, the original calibration procedure recorded targets from the full sequential forward pass but paired them with bridge *inputs* also taken from that same full pass, not from the actual distorted states Hop6’s skip-path produces. That is, the bridge was trained on clean, in-distribution inputs and deployed on noisy, skip-distorted inputs it had never seen – a standard train/test distribution mismatch.

Fix. Calibration now performs two forward passes per training sample:

1. Full model, all layers, sequential $\rightarrow$ records h_k^{full} for every layer boundary k (targets).
2. Hop6’s actual skip-path, wormholes active, bridge disabled $\rightarrow$ records the hidden state arriving at each wormhole destination (bridge inputs).

The indexing convention was verified explicitly: for a wormhole edge (i, j) , the bridge receives h_i^{hop6} (the state *after* layer i runs) and is trained toward h_{j-1}^{full} (the state *after* layer $j - 1$ runs in the full pass, i.e. what layer j is trained to receive as input).

3.3 Isolation testing

No mechanism existed to distinguish “the bridge is bad” from “routing or KV-cache handling is broken.” A diagnostic mode was added that runs Hop6 with the bridge fully disabled, a fixed deterministic path, and greedy decoding, so that raw skip-routing quality can be inspected in isolation. Given the bridge’s near-zero-contribution gate initialization (Section 2), an untrained bridge is already close to this condition, so this test also serves as a sanity check on whether the originally observed gibberish was attributable to the bridge at all, or was already present in uncorrected skip-routing.

3.4 Status

The graph non-determinism and distribution-mismatch fixes are implemented. The KV-cache / paging hypothesis has not yet been ruled in or out and remains the leading candidate if degenerate output persists after these fixes. No perplexity or output-quality numbers are reported here, since retraining on the corrected pipeline had not yet been run to convergence at time of writing.

4 Proposed Extensions

Three extensions to the routing mechanism are proposed. For each, related prior work is cited explicitly, and the discussion separates the general mechanism (not novel) from the specific way it would attach to Hop6’s graph structure (the part, if any, that would be).

4.1 Confidence-based early halting

Mechanism. Rather than always routing a token along a fixed-length path, attach a halting signal that allows the token to stop early once a confidence threshold is reached.

Prior art. Adaptive Computation Time [1] trains a halting unit to decide, at each recurrent step, whether to continue computing, with a ponder cost penalizing extra steps. Confident Adaptive Language Modeling (CALM) [2] applies essentially this idea to transformer decoding directly: a per-layer confidence measure determines whether a token exits the layer stack early. Neither uses graph-routing; both use a linear (fixed-order) layer stack.

Proposed Hop6-specific piece. Rather than halting a linear walk through layers, halting would occur *mid-graph-search*: a token’s hop budget would no longer be fixed (e.g. always ≈ 6 hops), but would terminate once a learned confidence signal – potentially derived from how much correction the bridge network is still applying, since large bridge corrections plausibly indicate the token is still far from a stable representation – crosses a threshold.

Known risk. Confidence estimates in early-exit systems are known to be miscalibrated precisely on the inputs where correctness matters most (rare tokens, negation, numerical content);

stacking this on top of an already lossy skip-routing scheme compounds approximation error and makes failures harder to attribute to a single cause.

4.2 Information-bottleneck routing

Mechanism. Instead of passing the full hidden-state vector across a wormhole edge, compress it to a smaller representation that retains only what the destination layer needs, discarding redundant or positional information not required downstream.

Prior art. This is the Information Bottleneck principle [3] applied at the level of inter-layer communication; it is directly related to bottleneck/compressed-communication layers used elsewhere in efficient-transformer literature.

Proposed Hop6-specific piece. Applying the compression *at the wormhole edges themselves*, so that a wormhole edge is not just a topology choice (“skip to layer j ”) but also a lossy channel with a bandwidth budget. The graph search would then need to trade off path length against information retained, rather than optimizing hop count alone. This combination of compression-as-edge-property with graph shortest-path search is, to the author’s knowledge, not present in existing information-bottleneck or early-exit work, neither of which has a graph-routing structure to attach compression to.

4.3 Graph-attached long-term memory

Mechanism. Extend effective context length by compressing tokens that would otherwise fall out of the context window into a smaller persistent memory, rather than discarding them.

Prior art. The Compressive Transformer [4] compresses discarded short-term memory into a fixed-size compressed memory bank via a learned compression function. The Memorizing Transformer [5] instead retrieves from a large cached key-value store via k -nearest-neighbors. Recurrent Memory Transformer architectures pass small sets of learned “memory tokens” across segments. All extend usable context without literal full-resolution attention over the entire history.

Proposed Hop6-specific piece. Rather than a flat memory module bolted onto the architecture, compressed memory slots would be reachable as another destination type for wormhole edges – so a token’s routing decision could jump not only to a different layer, but to a compressed memory slot from earlier context, using the same graph-search mechanism already used for layer-skipping.

Known risk – unbounded memory growth. Unlike the other two extensions, a memory module’s cost grows with conversation length unless explicitly capped. The Compressive Transformer avoids unbounded growth with a fixed-size FIFO memory bank, evicting the oldest compressed entries once full; any Hop6 implementation would need an equivalent hard cap from the outset, not as a later fix once VRAM usage is observed to grow.

4.4 Combined resource cost, unverified

Fixed-weight overhead from all three extensions together is expected to be small relative to model size: the bridge network is already budgeted at roughly 5–20M parameters, and a halting head plus a bottleneck encoder/decoder pair are each of comparable or smaller order, so total added parameters are plausibly in the tens of millions – negligible next to a multi-billion parameter base model. Activation memory added per forward pass (confidence scores, compressed vectors, memory read/write buffers) is expected to be on the order of tens of megabytes. The one component without a natural memory ceiling is uncapped long-term memory (Section 4.3); with a fixed-size cap it should also be bounded. *These figures are estimates, not measurements; they should be confirmed*

with direct profiling (parameter counts and `torch.cuda.max_memory_allocated()` peak activation memory) once any extension is implemented, rather than assumed.

5 Limitations

- No end-to-end perplexity or output-quality numbers are reported in this document. The bridge-training pipeline has been corrected but not yet retrained to convergence and re-evaluated.
- The KV-cache / paging hypothesis for the originally observed gibberish has not been ruled out; if degenerate output persists after the fixes in Section 3, this is the leading remaining candidate.
- None of the three extensions in Section 4 has been implemented or benchmarked. Each stacks additional approximation error (bridge correction, halting confidence, compression loss, memory staleness) on top of a system whose baseline quality is not yet confirmed. Extension work should not begin until the corrected bridge-training pipeline produces finite perplexity and coherent baseline output.
- The claim of novelty in Section 4 is limited specifically to the combination of each mechanism with graph-based layer routing; the underlying mechanisms themselves (halting, bottleneck compression, compressed memory) are established prior art and are cited as such.

References

- [1] A. Graves. *Adaptive Computation Time for Recurrent Neural Networks*. arXiv:1603.08983, 2016.
- [2] T. Schuster, A. Fisch, J. Gupta, et al. *Confident Adaptive Language Modeling*. NeurIPS, 2022.
- [3] N. Tishby, F. C. Pereira, W. Bialek. *The Information Bottleneck Method*. arXiv:physics/0004057, 2000.
- [4] J. W. Rae, A. Potapenko, S. M. Jayakumar, T. P. Lillicrap. *Compressive Transformers for Long-Range Sequence Modelling*. arXiv:1911.05507, 2019.
- [5] Y. Wu, M. N. Rabe, D. Hutchins, C. Szegedy. *Memorizing Transformers*. arXiv:2203.08913, 2022.